



Tao Hyperliquid Bridge Audit Report

Prepared by [Burra Security](#)

Version 1.1

Researchers

wirew0lf

santipu03

IvanFitro

April 7, 2026

Contents

1	About Burra Security	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	6
7.1	Medium Risk	6
7.1.1	Deployer Retains LayerZero Delegate Authority After Ownership Transfer	6
7.1.2	RelayComposer contract balance can be drained via low minRelayFee	6
7.1.3	RelayComposer does not refund msg.value on failed relays	7
7.1.4	RelayComposer refunds are lost when the sender cannot access the relay chain	8
7.1.5	DoS on most MyNativeOFTAdapter functionality due to high-level calls to precompiles	9
7.2	Low Risk	12
7.2.1	OFTReceived event emits wrong address for substrate recipients	12
7.2.2	HyperLiquidComposer refundToSrc sends tokens to RelayComposer instead of the user	12
7.2.3	Ownership transfer from the deployer EOA to governance is not enforced by the deployment pipeline	13
7.2.4	No Programmatic Post-Deployment Ownership Verification	14
7.2.5	Mainnet enforced compose gas insufficient for dummy hllIndexId refund path	14
7.2.6	LayerZero wiring depends on unpinned live metadata	15
7.2.7	Auto-unstake in _lzReceive() blocked by staking rate limit	15
7.2.8	setStakingConfig allows switching to non-root subnets	16
7.2.9	lzCompose() does not refund excess fees	16
7.2.10	setStakingConfig() missing check for same values	18
7.3	Informational	19
7.3.1	The safe post-handoff workflow cannot run from the checked-in config	19
7.3.2	Single-DVN Verification Model Creates a Centralized Trust Assumption	19
7.3.3	HyperEVM deploy scripts break if MNEMONIC is used instead of a raw PRIVATE_KEY	19
7.3.4	Environment-Specific Secret Files Are Not Git Ignored	20
7.3.5	Missing explicit post-unstake liquidity check in _lzReceive()	20
7.3.6	LayerZero shared-decimal quantization introduces a minimum effective bridge amount	21
7.3.7	registerClaimProxy missing msg.value validation	21

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The HTAO Bridge locks native TAO on Bittensor and uses LayerZero V2's OFT standard with a relay composer to hop messages through to HyperEVM. The owner/operator can stake locked TAO on Bittensor to earn yield, while the bridge maintains a solvency invariant ensuring total deposits never exceed the available plus staked balance. Staking is controlled by privileged roles, and HTAO itself is not a yield-bearing token.

5 Audit Scope

Everything under <https://github.com/0x-whisperer/tao-hyperliquid-bridge>.

6 Executive Summary

Over the course of 4 days, the Burra Security team conducted an audit on the [Tao Hyperliquid Bridge](#) smart contracts provided by [Mentat Minds](#). In this period, a total of 22 issues were found.

Key focus areas:

1. Bittensor Precompile Interactions

The adapter makes direct calls to 4 Bittensor precompiles (Staking V2, Address Mapping, Proxy, Balance Transfer) at hardcoded addresses. These are non-standard targets, no deployed bytecode, custom runtime behavior, and Substrate-level side effects invisible to the EVM. Every staking operation, bridge-out credit, and proxy registration depends on these calls behaving as expected across compiler versions and chain upgrades.

2. Three-Chain Relay Architecture

The bridge routes messages through an intermediary chain (Bittensor → Arbitrum → HyperEVM) because direct LayerZero routing isn't available. This doubles the cross-chain attack surface: two hops of message encoding/decoding, two sets of enforced options, two fee quotes, and a transient mint-then-burn on the relay chain. Each hop introduces points where messages can fail, be mispriced, or be composed incorrectly.

3. Native Token Locking & Solvency Invariant

MyNativeOFTAdapter locks native TAO (not ERC20) and tracks obligations via a single totalDeposited counter. The solvency invariant ($\text{totalDeposited} \leq \text{liquid balance} + \text{staked balance}$) must hold across every code path, bridge-in, bridge-out, staking, unstaking, yield harvesting, and failure recovery. Any path that moves TAO without correctly updating this counter risks insolvency.

4. Staking of User Deposits for Yield

The owner/operator stakes locked user deposits on Bittensor's root subnet to earn yield. This creates a dual-balance system (liquid + staked) where bridge-out must potentially unstake to service withdrawals. The risk surface includes subnet economics (root vs non-root behavior), rate-limiting constraints, dividend mechanics, and the fact that staking decisions are made by a privileged role over funds that belong to bridge users.

5. Relay Failure & Refund Paths

When relay forwarding fails (missing peer, insufficient fee, compose revert), the system must gracefully refund users. These failure paths involve refunding tokens on an intermediary chain the user may never directly interact with, handling native value (ETH) separately from token balances, and correctly identifying the original sender across chain boundaries.

6. Compose Message Propagation

LayerZero compose messages carry instructions beyond simple token transfers, routing to HyperCore spot balances, specifying final destinations, encoding minimum fees. These payloads are encoded on the source chain, decoded on the relay, potentially re-encoded for hop 2, and finally executed on the destination. Each boundary is a parsing and validation surface where malformed, unexpected, or attacker-crafted payloads must be handled safely.

7. Decimal & Unit Conversion Boundaries

TAO uses 9-decimal rao natively on Bittensor but is represented as 18-decimal wei in the EVM. Every precompile interaction requires conversion at the boundary. Rounding, truncation, and dust accumulation across repeated operations are inherent risks in a system that constantly crosses this 9-decimal gap.

8. Access Control & Privileged Operations

The trust model splits privileges between owner and operator, with the owner controlling configuration, proxy registration, and fund recovery, while the operator handles day-to-day staking. Recovery functions (`recover()`) grant broad power over stuck funds. The security boundary between "yield earned by the protocol" and "deposits owed to users" is enforced purely by accounting logic, not by fund segregation.

Final assessment:

After reviewing the fixes, we rate our confidence at 9/10. We were able to verify the major components end-to-end and don't see any serious issues remaining. The main reason it isn't higher is the added complexity of Bittensor and Hyperliquid, where chain-specific edge cases are always a possibility. Aside from that, the codebase is fairly straightforward and closely follows LayerZero templates with minimal changes.

Summary

Project Name	Tao Hyperliquid Bridge
Repository	tao-hyperliquid-bridge
Commit	2b618f81f6fa...
Audit Timeline	March 25th - March 30th
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	5
Low Risk	10
Informational	7
Gas Optimizations	0
Total Issues	22

Summary of Findings

[M-1] Deployer Retains LayerZero Delegate Authority After Ownership Transfer	Resolved
[M-2] RelayComposer contract balance can be drained via low minRelayFee	Resolved
[M-3] RelayComposer does not refund msg.value on failed relays	Resolved
[M-4] RelayComposer refunds are lost when the sender cannot access the relay chain	Resolved
[M-5] DoS on most MyNativeOFTAdapter functionality due to high-level calls to precompiles	Resolved
[L-1] OFTReceived event emits wrong address for substrate recipients	Resolved
[L-2] HyperLiquidComposer refundToSrc sends tokens to RelayComposer instead of the user	Acknowledged
[L-3] Ownership transfer from the deployer EOA to governance is not enforced by the deployment pipeline	Resolved
[L-4] No Programmatic Post-Deployment Ownership Verification	Resolved
[L-5] Mainnet enforced compose gas insufficient for dummy hlIndexId refund path	Acknowledged
[L-6] LayerZero wiring depends on unpinned live metadata	Acknowledged
[L-7] Auto-unstake in _lzReceive() blocked by staking rate limit	Acknowledged
[L-8] setStakingConfig allows switching to non-root subnets	Resolved
[L-9] lzCompose() does not refund excess fees	Acknowledged
[L-10] setStakingConfig() missing check for same values	Resolved
[I-1] The safe post-handoff workflow cannot run from the checked-in config	Acknowledged
[I-2] Single-DVN Verification Model Creates a Centralized Trust Assumption	Acknowledged
[I-3] HyperEVM deploy scripts break if MNEMONIC is used instead of a raw PRIVATE_KEY	Acknowledged
[I-4] Environment-Specific Secret Files Are Not Git Ignored	Resolved
[I-5] Missing explicit post-unstake liquidity check in _lzReceive()	Acknowledged

[I-6] LayerZero shared-decimal quantization introduces a minimum effective bridge amount	Acknowledged
[I-7] registerClaimProxy missing msg.value validation	Resolved

7 Findings

7.1 Medium Risk

7.1.1 Deployer Retains LayerZero Delegate Authority After Ownership Transfer

Target: [MyOFT.sol](#), [MyNativeOFTAdapter.sol](#), [MyOFT.ts](#), [MyNativeOFTAdapter.ts](#), [layerzero.mainnet.config.ts](#)

Severity:

- Impact: High
- Likelihood: Medium

Description: The deployment flow in [MyOFT.ts:73](#) and [MyNativeOFTAdapter.ts:27](#) passes the deployer EOA into the constructors of [MyOFT.sol:8](#) and [MyNativeOFTAdapter.sol:114](#), making the deployer the initial LayerZero delegate.

However, the post-deployment configuration in [layerzero.mainnet.config.ts:97](#) and [layerzero.testnet.config.ts:97](#) only specifies owner, not delegate, and the hardening procedure in [greenfield-deployment.md:387](#) only instructs operators to transfer ownership and verify `owner()`.

This means the system can appear multisig-owned while the original deployer still retains LayerZero configuration authority over messaging libraries, DVN, and executor settings. If that deployer key is later compromised, an attacker may be able to reconfigure cross-chain verification/execution and ultimately steal the TAO locked in the Bittensor adapter.

Recommended Mitigation: Explicitly rotate the LayerZero delegate for every OApp to the intended governance address during the post-deployment hardening process, and verify both owner and delegate onchain before considering the deployer neutralized. The LayerZero config files should include the intended delegate state, and deployment procedures should require validating delegate rotation in addition to `owner()` checks.

Client: Fixed in commit [ec8bede](#)

BurraSec: Fix looks good

7.1.2 RelayComposer contract balance can be drained via low minRelayFee

Target: [RelayComposer.sol](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: The `RelayComposer.lzCompose` function uses the contract's own ETH balance to pay the LayerZero fee for hop 2 of the relay. The `minRelayFee` value, which is decoded from the user-controlled compose message, only serves as a minimum threshold for the compose value delivered by the executor:

```
if (minRelayFee > 0) {
    require(msg.value >= minRelayFee, "RelayComposer: insufficient relay fee");
}

// ...

MessagingFee memory fee = oft.quoteSend(sendParam, false);
if (address(this).balance < fee.nativeFee) {
    // refund path
}

oft.send{value: fee.nativeFee}(sendParam, fee, address(this));
```

Since `minRelayFee` is entirely user-specified and the fee payment draws from `address(this).balance` (which includes both `msg.value` and any pre-existing balance), a user can set `minRelayFee` to 0 and have the contract

pay for their relay using ETH that was already sitting in the contract. Any excess ETH accumulated from previous transactions where `minRelayFee` exceeded the actual LayerZero fee can be consumed by subsequent relays that specify a lower (or zero) `minRelayFee`.

The practical impact is limited because under normal operation the contract should not hold significant excess ETH. However, ETH can accumulate when `minRelayFee` consistently overestimates the actual LayerZero fee across multiple transactions, or from the separate issue where `msg.value` is not refunded on failed relays.

Recommended Mitigation: Instead of checking `address(this).balance` against the quoted fee, validate that `msg.value` alone is sufficient to cover it. This ensures each relay is self-funded and the contract never subsidizes a relay with leftover ETH from prior transactions:

```
MessagingFee memory fee = oft.quoteSend(sendParam, false);
- if (address(this).balance < fee.nativeFee) {
+ if (msg.value < fee.nativeFee) {
    IERC20(address(oft)).safeTransfer(sender, amountLD);
    emit RelayFailed(_guid, finalDstEid, amountLD, sender, "insufficient fee");
    return;
}
```

Client: Fixed in commit [9fe7f93](#)

BurraSec: Fix looks good, using `msg.value` instead of contract balance to verify.

7.1.3 RelayComposer **does not refund** `msg.value` **on failed relays**

Target: [RelayComposer.sol](#)

Severity:

- Impact: Medium
- Likelihood: Medium

Description: The `RelayComposer.lzCompose` function handles compose callbacks from the LayerZero Executor after tokens are minted to the contract. The function receives native ETH via `msg.value`, which represents the compose value delivered by the executor to fund hop 2 of the relay. There are two failure paths where the function refunds the minted tokens back to the original sender but returns early without handling the delivered `msg.value`:

```
if (IOAppCore(address(oft)).peers(finalDstEid) == bytes32(0)) {
    IERC20(address(oft)).safeTransfer(sender, amountLD);
    emit RelayFailed(_guid, finalDstEid, amountLD, sender, "no peer");
    return;
}

// ...

MessagingFee memory fee = oft.quoteSend(sendParam, false);
if (address(this).balance < fee.nativeFee) {
    IERC20(address(oft)).safeTransfer(sender, amountLD);
    emit RelayFailed(_guid, finalDstEid, amountLD, sender, "insufficient fee");
    return;
}
```

In both cases, the `msg.value` that was delivered by the executor as compose value is left in the `RelayComposer` contract. Since the caller is the LayerZero Endpoint, the native tokens should be sent back to the executor or the original sender, but instead they silently accumulate in the contract.

The impact is that every time a relay fails due to a missing peer or insufficient fee balance, the compose value paid by the original sender on the source chain is effectively lost. These funds can only be retrieved through the owner-gated `recover` function, requiring manual intervention and trust in the owner to return them.

Recommended Mitigation: Refund the `msg.value` back to the executor (or the original sender) in both failure paths.

Client: Fixed in commit [e4d049c](#)

BurraSec: Fix looks good, refund native tokens to sender on relay failure

7.1.4 RelayComposer refunds are lost when the sender cannot access the relay chain

Target: [RelayComposer.sol](#)

Severity:

- Impact: High
- Likelihood: Medium

Description: When the RelayComposer receives tokens via a compose callback and the relay forward fails, the contract refunds the tokens to the original sender directly on the relay chain, which will be Arbitrum. This happens in two failure paths: when the destination has no peer configured, and when the contract lacks sufficient native fee balance. In both cases, the refund is a local `safeTransfer` to the sender address on the relay chain.

```
function lzCompose(
    address _from,
    bytes32 _guid,
    bytes calldata _message,
    address _executor,
    bytes calldata _extraData
) external payable {
    // ...
>>    address sender = address(uint160(uint256(OFTComposeMsgCodec.composeFrom(_message))));
    // ...

    // ...
    if (IOAppCore(address(of)).peers(finalDstEid) == bytes32(0)) {
>>        IERC20(address(of)).safeTransfer(sender, amountLD);
        emit RelayFailed(_guid, finalDstEid, amountLD, sender, "no peer");
        return;
    }

    // ...

    // 3. Quote fee for hop 2 and check balance
    MessagingFee memory fee = oft.quoteSend(sendParam, false);
    if (address(this).balance < fee.nativeFee) {
>>        IERC20(address(of)).safeTransfer(sender, amountLD);
        emit RelayFailed(_guid, finalDstEid, amountLD, sender, "insufficient fee");
        return;
    }

    // ...
}
```

The sender address is the original caller on the source chain (e.g., Bittensor). If that sender is a smart account, multisig, or any contract-based wallet, the same address likely does not exist or is not accessible on the relay chain (e.g., Arbitrum). In that case, the refunded tokens are permanently locked since no one controls that address on the relay chain.

The impact is a complete loss of the bridged tokens for any non-EOA sender whenever a relay failure occurs. Even for EOA senders, relying on them having access to the relay chain is a fragile assumption since the relay chain is an intermediary that users may never interact with directly.

Recommended Mitigation: Instead of transferring tokens locally on the relay chain, bridge the refund back to the source chain using `OFT.send`. This is the pattern used by LayerZero's own `VaultComposerSync._refund`, which extracts the source chain EID from the compose message and sends the tokens back to the original sender on their origin chain.

A simpler alternative, if bridge-back complexity is undesired, would be to hold the tokens in the `RelayComposer` contract itself and emit an event with the sender and amount details, allowing the owner to manually recover and return them via the existing `recover` function. However, the bridge-back approach is strictly safer and does not rely on admin intervention.

Client: Fixed in commit [e4d049c](#) and [6d46bbb](#)

BurraSec: Fix looks good, override `send()` in `MyOFT` and `MyNativeOFTAdapter` to enforce `tx.origin == msg.sender`, restricting usage to EOAs only.

7.1.5 DoS on most `MyNativeOFTAdapter` functionality due to high-level calls to precompiles

Target: `MyNativeOFTAdapter`

Severity:

- Impact: Medium
- Likelihood: High

Description: `MyNativeOFTAdapter` interacts with four Bittensor precompiles (Staking V2, Address Mapping, Proxy, and Balance Transfer) through Solidity interface types cast to constant addresses. These precompiles are implemented at the Substrate runtime level and do not have EVM bytecode deployed at their addresses.

Since Solidity 0.8.10, the compiler inserts an `EXTCODESIZE` check before executing external calls to functions that return no value (void functions). If the target address has zero code size, the call reverts before the `CALL` opcode is even executed. This behavior is documented in the [official Bittensor EVM examples](#), which explicitly warn against high-level calls:

```
// can't call precompile like this way, the call never go to runtime precompile
// Staking(ISTAKING_ADDRESS).addStake(hotkey, amount, netuid);
```

The contract uses `pragma solidity ^0.8.22` and makes five high-level calls to void precompile functions:

```
STAKING.addStake(hotkey, amountRao, netuid);           // L150
STAKING.removeStake(hotkey, amountRao, netuid);        // L162, L200, L275
PROXY.addProxy(proxyAccountId32, 17, 0);              // L214
PROXY.removeProxy(proxyAccountId32, 17, 0);           // L221
BALANCE_TRANSFER.transfer{value: amountLD}(toAccountId32); // L285
```

All five calls will revert due to the `EXTCODESIZE` check, breaking `stakeBalance()`, `unstakeBalance()`, `setStakingConfig()`, `registerClaimProxy()`, `removeClaimProxy()`, and most critically `_lzReceive()`. The two calls with return values (`getStake` and `addressMapping`) are not affected because the compiler skips the code-size check for functions with return data, relying instead on the ABI decoder.

The impact is a complete DoS on the contract. Staking, unstaking, proxy management, and bridge-out operations are all non-functional.

PoC:

To run the following test, use the command:

```
forge test --mc BittensorPoC --evm-version london -vvv
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.22;

import "lib/forge-std/src/Test.sol";
```

[illegible]

```

    vm.expectRevert();
    caller.callTransfer{value: 1e18}(BALANCE_TRANSFER, exampleDestination);
}

function test_lowLevelCallsSucceed() public {
    uint256 amount = 10e9;

    (bool success,) = address(STAKING).call(
        abi.encodeWithSelector(ISTakingV2.addStake.selector, exampleHotKey, amount, exampleNetUid)
    );
    require(success);

    (success,) = address(PROXY).call(
        abi.encodeWithSelector(IProxy.addProxy.selector, exampleProxy, 17, 0)
    );
    require(success);

    (success,) = address(BALANCE_TRANSFER).call{value: 1e18}(
        abi.encodeWithSelector(IBalanceTransfer.transfer.selector, exampleDestination)
    );
    require(success);
}

contract Caller {
    function callAddStake(ISTakingV2 staking, bytes32 hotkey, uint256 amount, uint16 netuid) external {
        staking.addStake(hotkey, amount, netuid);
    }

    function callAddProxy(IProxy proxy, bytes32 delegate, uint8 proxyType, uint32 delay) external {
        proxy.addProxy(delegate, proxyType, delay);
    }

    function callTransfer(IBalanceTransfer bt, bytes32 dest) external payable {
        bt.transfer{value: msg.value}(dest);
    }
}

```

Recommended Mitigation: Replace all void precompile calls with low-level `.call()` using `abi.encodeWithSelector`, matching the pattern used in the [official Bittensor examples](#):

```

(bool success, ) = address(STAKING).call(
    abi.encodeWithSelector(ISTakingV2.addStake.selector, hotkey, amountRao, netuid)
);
require(success, "addStake failed");

```

The same pattern should be applied to all five affected call sites: `addStake`, `removeStake`, `addProxy`, `removeProxy`, and `BALANCE_TRANSFER.transfer`.

Client: Fixed in commit [a44ed03](#)

BurraSec: Fix looks good, using low level calls to call precompiles.

7.2 Low Risk

7.2.1 OFTReceived event emits wrong address for substrate recipients

Target: [MyNativeOFTAdapter.sol](#).

Severity:

- Impact: Low
- Likelihood: High

Description: When a cross-chain transfer is received in `MyNativeOFTAdapter._lzReceive()`, the function credits the recipient via the Balance Transfer Precompile using the raw `bytes32 toAccountId32` from the LayerZero message. This `toAccountId32` can represent either a native Substrate account (SS58 public key) or a derived EVM account (`blake2_256("evm:" || H160)`).

At the end of execution, the function emits the standard LayerZero `OFTReceived` event, which expects an address (20 bytes) as the recipient parameter. The current conversion truncates the 32-byte account ID to its lower 20 bytes:

```
emit OFTReceived(_guid, _origin.srcEid, address(uint160(uint256(toAccountId32))), amountLD);
```

When `toAccountId32` is a native Substrate account, the lower 20 bytes of the public key do not correspond to any meaningful EVM address. Similarly, when `toAccountId32` is a derived EVM account, the lower 20 bytes of the blake2 hash do not recover the original H160 address. In both cases, the emitted address is incorrect. This affects off-chain indexers, monitoring dashboards, and block explorers that rely on this event to track bridge recipients, potentially causing failed attribution of received funds.

The same incorrect conversion also appears for the zero-amount early return path.

Recommended Mitigation: Since the `OFTReceived` event is defined in LayerZero's IOFT interface with an address parameter, it cannot natively represent a 32-byte Substrate account. Consider emitting an additional custom event that logs the full `bytes32 toAccountId32`, so off-chain systems can accurately identify the actual recipient:

```
event OFTReceivedSubstrate(bytes32 indexed guid, uint32 srcEid, bytes32 indexed toAccountId32, uint256  
↳ amountReceivedLD);
```

For the standard `OFTReceived` event, consider emitting `address(0)` to avoid publishing misleading data.

Client: Fixed in commit [d1f299d](#)

BurraSec: Fix looks good

7.2.2 HyperLiquidComposer refundToSrc sends tokens to RelayComposer instead of the user

Target: [MyHyperLiquidComposer_Recoverable.sol](#).

Severity:

- Impact: Medium
- Likelihood: Low

Description: The `HyperLiquidComposer` is designed to handle refunds to the source chain when a composed message fails to decode. As its own NatSpec [states](#): *"This composer is designed to handle refunds to source to an EOA address and NOT a contract."*

When `decodeMessage` reverts, the contract stores a `FailedMessage` whose refund destination is derived from `composeFrom`, the address that called `OFT.send` on the source chain.

```
catch {  
    SendParam memory refundSendParam;  
    refundSendParam.dstEid = OFTComposeMsgCodec.srcEid(_message);  
    refundSendParam.to = OFTComposeMsgCodec.composeFrom(_message);  
    refundSendParam.amountLD = amountLD;
```

```
failedMessages[_guid] = FailedMessage({ refundSendParam: refundSendParam, msgValue: msg.value });
}
```

In the multi-hop flow (Bittensor -> Arbitrum -> HyperEVM), hop 2 is always initiated by RelayComposer calling `oft.send()` on Arbitrum. The OFT standard encodes `msg.sender` as the `composeFrom` field (via `OFTMsgCodec.encode`), so on HyperEVM `composeFrom` always resolves to the RelayComposer contract address, never the original user. This means the refund mechanism fundamentally cannot work as intended in this architecture, since the initiator is always a contract.

When `refundToSrc` is called, tokens are burned on HyperEVM and minted to the RelayComposer contract on Arbitrum. The original user's address from Bittensor is completely lost in the refund path. The tokens sit in RelayComposer as ERC20 (MyOFT), while the user's original TAO remains locked in MyNativeOFTAdapter on Bittensor. Recovery requires the owner to manually call `recover()`, identify the affected user through off-chain event tracing, and give the tokens back to the user.

Recommended Mitigation: Given that the root cause of the issue is in the inherited HyperLiquidComposer contract, we recommend acknowledging the issue, and accepting the admin-recovery path as a known operational requirement. The `FailedMessageDecode` event emitted on HyperEVM (which includes the guid) can be correlated off-chain with the original user's transaction on Bittensor via the RelayComposer's `compose` logs, allowing the owner to identify the affected user and return funds via `recover()`.

Client: Acknowledged, inherent to multi-hop architecture, recovery via `recover()`.

BurraSec: Acknowledged

7.2.3 Ownership transfer from the deployer EOA to governance is not enforced by the deployment pipeline

Target: [layerzero.mainnet.config.ts](#), [greenfield-deployment.md](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: The deployment flow does not enforce transfer of ownership from the deployer EOA to the intended Safe/governance addresses. Both LayerZero config files state that the configured owner field is ignored by `lz:deploy` and `lz:oapp:wire`, and the handoff only happens if operators later remember to run `lz:ownable:transfer-ownership`. If that manual step is skipped or partially executed, the deployer EOA retains owner privileges over the bridge's owned contracts, including peer/config rewiring and relay recovery powers.

Recommended Mitigation: Add an explicit post-deploy ownership-transfer step to the deployment pipeline and fail the deployment unless onchain `owner()` checks match the expected Safe addresses on every governed contract. The handoff should be treated as part of deployment completion, not as an optional later runbook step.

Client: Fixed in commit [bf0d6d9](#)

BurraSec: Fix looks good, added new `lz:check:hardening` verification task.

7.2.4 No Programmatic Post-Deployment Ownership Verification

Target: Deployment workflow (M3.2 — `lz:ownable:transfer-ownership`)

Severity:

- Impact: Medium
- Likelihood: Low

Description: After the M3.2 ownership transfer milestone, there is no automated verification script to confirm that all four deployed contracts have successfully transferred ownership to the intended Gnosis Safe addresses.

The existing `checkBridgeBalances.ts` task validates balances but does not check `owner()` on any contract.

Recommended Mitigation: Add a post-deployment verification Hardhat task that reads `owner()` from all four contracts across all three chains and asserts they match the expected `SAFE_*` addresses from `.env`. Run this as a mandatory step after M3.2 and before the deployer key is neutralized. Include a check for `RECOVERY_ADDRESS` on the `Composer` and `ConvertToMultiSigUser` status on `HyperCore`.

Client: Fixed in the commit [bf0d6d9](#)

BurraSec: Fix looks good, added new `lz:check:hardening` verification task.

7.2.5 Mainnet enforced compose gas insufficient for dummy `hlIndexId` refund path

Target: [layerzero.mainnet.config.ts](#), [MyHyperLiquidComposer_Recoverable.ts](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: The composer deploy script accepts `coreSpotIndex = none` on any network, deploying with a dummy `hlIndexId = 99999` intended as a "graceful refund mode." The testnet config allocates 5M compose gas for this scenario because the `CoreWriter` precompile consumes ALL forwarded gas on an invalid index, and the EIP-150 1/64th gas retention rule must leave enough for the catch block's `safeTransfer` refund (~60k gas). However, the mainnet config enforces only 250k compose gas — calibrated for a valid `hlIndexId` where `CoreWriter` returns unused gas on failure.

If the mainnet composer is deployed with `none` before the HIP-1 token is finalized, every compose transfer reverts.

The compose message remains retryable in the `EndpointV2` queue, and a manual retry with a higher transaction gas limit succeeds. The `RECOVERY_ADDRESS` can also sweep stuck tokens via `recoverEvmERC20()`. However, the automated executor delivery fails on every attempt since it always uses the 250k enforced budget, silently degrading every user's compose transfer until someone intervenes manually.

Recommended Mitigation: Gate the `none` path behind a mainnet refusal in the deploy script, or emit a prominent warning that the enforced compose gas must be raised to match the testnet's 5M budget before wiring. Alternatively, add a mainnet-specific check in the wiring config that detects `hlIndexId = 99999` and overrides compose gas to 5M automatically.

Client: Acknowledged, Dummy `hlIndexId` not used on mainnet.

BurraSec: Acknowledged

7.2.6 LayerZero wiring depends on unpinned live metadata

Target: [layerzero.mainnet.config.tsl](#), [layerzero.testnet.config.ts](#)

Severity:

- Impact: High
- Likelihood: Low

Description: LayerZero config files call `generateConnectionsConfig(pathways)` without supplying a custom `fetchMetadata`, so they rely on `@layerzerolabs/metadata-tools` default behavior to fetch live metadata at execution time before generating the `lz:oapp:wire` transaction set.

The repo does contain documented expected addresses in `layerzero.md`, but those values are not enforced or checked programmatically before wiring. As a result, any rewiring or emergency config change depends on a live off-repo metadata source for security-critical trust configuration. If that source is stale, poisoned, or overridden via `LZ_METADATA_URL`, operators can generate and potentially execute a malicious or bricking config update.

Recommended Mitigation: Pin the expected LayerZero send library, receive library, executor, and DVN addresses per network in-repo and stop relying on live metadata for production wiring. At minimum, add a hard assertion layer that compares every resolved library/DVN/executor address against an in-repo allowlist before `generateConnectionsConfig()` output is used.

Client: Acknowledged

BurraSec: Acknowledged

7.2.7 Auto-unstake in `_lzReceive()` blocked by staking rate limit

Target: [MyNativeOFTAdapter.sol](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: the Bittensor staking precompile enforces a 1-operation-per-block rate limit per `(hotkey, coldkey, netuid)` tuple. If any staking operation (`stakeBalance()`, `unstakeBalance()`, `setStakingConfig()`, or a previous auto-unstake) was executed in the same block, the `STAKING.removeStake()` call will revert, causing the entire `_lzReceive()` to revert. The bridge-out message enters the LayerZero retry queue.

In a pathological scenario where the operator has an automated staking bot that stakes every block, bridge-outs requiring auto-unstake would fail repeatedly. User funds are not lost (messages are retryable), but user experience degrades.

Recommended Mitigation: Maintain a liquid TAO buffer in the adapter and consider preventing `stakeBalance(0)` from staking the full balance. Add a configurable `minLiquidBalance` so most bridge-outs can complete without auto-unstake, and coordinate operator staking actions to avoid same-block collisions with bridge-out retries.

Client: Acknowledged, Bittensor rate limit enforced, messages are retryable via LayerZero.

BurraSec: Acknowledged

7.2.8 setStakingConfig allows switching to non-root subnets

Target: [MyNativeOFTAdapter.sol](#)

Severity:

- Impact: High
- Likelihood: Low

Description: The contract is designed to stake exclusively on the root subnet (netuid 0), where alpha equals TAO at a 1:1 ratio. However, setStakingConfig accepts an arbitrary _netuid parameter with no validation, allowing the operator or owner to switch to any non-root subnet.

```
function setStakingConfig(bytes32 _hotkey, uint256 _netuid) external onlyOperatorOrOwner {
    uint256 staked = stakedBalance();
    if (staked > 0) {
        uint256 stakedRao = staked / WEI_PER_RAO;
        STAKING.removeStake(hotkey, stakedRao, netuid);
        emit Unstaked(stakedRao * WEI_PER_RAO);
    }
    hotkey = _hotkey;
    netuid = _netuid;
    emit StakingConfigUpdated(_hotkey, _netuid);
}
```

On non-root subnets, TAO is converted to alpha tokens through an AMM, which introduces multiple accounting issues: stakedBalance() would return alpha-denominated values instead of TAO, removeStake amounts are interpreted as alpha rather than rao, and unstaking is subject to AMM slippage. These break the contract's core solvency invariant (totalDeposited <= balance + stakedBalance), as totalDeposited is always in TAO while stakedBalance() would silently return a value in a different denomination.

Since the contract is only intended for root subnet staking, this unnecessary flexibility introduces risk without any benefit.

Recommended Mitigation: Restrict setStakingConfig to only allow changing the hotkey, and hardcode netuid to 0. Alternatively, add some comments to make future owner/operator aware of this issue.

Client: Fixed in commit [9eee050](#)

BurraSec: Fix looks good, hardcode netuid to root subnet (0)

7.2.9 lzCompose() does not refund excess fees

Target: RelayComposer.sol

Severity:

- Impact: Low
- Likelihood: Medium

Description: In lzCompose() a user (via the executor) sends msg.value to cover the fee.nativeFee required for the LayerZero transaction.

```
function lzCompose(
    address _from,
    bytes32 _guid,
    bytes calldata _message,
    address _executor,
    bytes calldata _extraData
) external payable {
    require(msg.sender == endpoint, "!endpoint");
    require(_from == address(oft), "!oft");
```

```

// 1. Decode compose message
// - amountLD: how many tokens were minted to us
// - composeFrom: original sender on the source chain (for refunds)
// - composeMsg: (uint32 finalDstEid, bytes32 toAddress, bytes composeForHop2, uint256 minRelayFee)
uint256 amountLD = OFTComposeMsgCodec.amountLD(_message);
address sender = address(uint160(uint256(OFTComposeMsgCodec.composeFrom(_message))));
(uint32 finalDstEid, bytes32 toAddress, bytes memory composeForHop2, uint256 minRelayFee) =
    abi.decode(OFTComposeMsgCodec.composeMsg(_message), (uint32, bytes32, bytes, uint256));

// 1b. Validate executor delivered the agreed-upon compose value
if (minRelayFee > 0) {
    require(msg.value >= minRelayFee, "RelayComposer: insufficient relay fee");
}

// 1c. Validate destination has a peer set.
// Without this check, quoteSend() reverts with NoPeer and the compose entry
// stays in the endpoint queue permanently retryable but always failing.
// Graceful: refund tokens to the original sender on the relay chain.
if (IOAppCore(address(oft)).peers(finalDstEid) == bytes32(0)) {
    IERC20(address(oft)).safeTransfer(sender, amountLD);
    emit RelayFailed(_guid, finalDstEid, amountLD, sender, "no peer");
    return;
}

// 2. Build SendParam for hop 2, second call to LZ to send it to HyperCore
SendParam memory sendParam = SendParam({
    dstEid: finalDstEid,
    to: toAddress,
    amountLD: amountLD,
    minAmountLD: amountLD, // exact: amount is already dust-free from hop 1
    extraOptions: "", // Use enforced options
    composeMsg: composeForHop2, // Forward compose to destination (empty = no compose)
    oftCmd: ""
});

// 3. Quote fee for hop 2 and check balance
MessagingFee memory fee = oft.quoteSend(sendParam, false);

if (address(this).balance < fee.nativeFee) {
    IERC20(address(oft)).safeTransfer(sender, amountLD);
    emit RelayFailed(_guid, finalDstEid, amountLD, sender, "insufficient fee");
    return;
}

// 4. Send tokens to final destination
oft.send{value: fee.nativeFee}(sendParam, fee, address(this));
}

```

The issue is that the function lacks a refund mechanism for excess native tokens. If a user provides a `msg.value` that is greater than the `fee.nativeFee` quoted by `oft.quoteSend()`, the surplus is not returned to the sender. Instead, it remains trapped in the contract. This results in users being overcharged for their cross-chain transactions without any way to recover the difference.

Recommended Mitigation: Create a mapping to track excess fees paid by each sender. When `msg.value` exceeds `fee.nativeFee`, store the surplus in the mapping and provide a function that allows users to withdraw their excess funds.

Client: Acknowledged

BurraSec: Acknowledged

7.2.10 setStakingConfig() missing check for same values

Target: [MyNativeOFTAdapter.sol](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: setStakingConfig() updates the staking parameters by first unstaking all assets from the current validator before setting the new hotkey and netuid.

```
function setStakingConfig(bytes32 _hotkey, uint256 _netuid) external onlyOperatorOrOwner {
    uint256 staked = stakedBalance();
    if (staked > 0) {
        uint256 stakedRao = staked / WEI_PER_RAO;
        STAKING.removeStake(hotkey, stakedRao, netuid);
        emit Unstaked(stakedRao * WEI_PER_RAO);
    }
    hotkey = _hotkey;
    netuid = _netuid;
    emit StakingConfigUpdated(_hotkey, _netuid);
}
```

The issue is that the function lacks a check to verify if the new configuration differs from the current one. If setStakingConfig() is called with the existing hotkey and netuid, it triggers an unnecessary unstaking process.

Recommended Mitigation: To fix this, add a check to ensure the new configuration differs from the current one.

Client: Fixed in commit [96559ee](#)

BurraSec: Fix looks good

7.3 Informational

7.3.1 The safe post-handoff workflow cannot run from the checked-in config

Target: [hardhat.config.ts](#)

Severity:

- Impact: Informational
- Likelihood: Low

Description: The documented post-handoff governance workflow depends on running `lz:oapp:wire --safe`, but the checked-in config does not define `safeConfig` for any network. As a result, the Safe-based smoke test described in the deployment guide cannot be executed from the repository as provided, which increases the chance that multisig operations fail during live ownership handoff or later emergency/configuration changes.

Recommended Mitigation: Add complete `safeConfig` entries for every governed network and validate them as part of the deployment checklist before ownership transfer.

Client: Acknowledged, the configuration is applied at handoff time rather than being set prematurely.

BurraSec: Acknowledged

7.3.2 Single-DVN Verification Model Creates a Centralized Trust Assumption

Target: [layerzero.mainnet.config.ts](#)

Severity:

- Impact: Informational
- Likelihood: Low

Description: All configured LayerZero pathways use `[['LayerZero Labs'], []]`, meaning one required DVN and no optional DVNs for message verification on the Bittensor/Arbitrum/HyperEVM routes and their testnet equivalents. Under LayerZero's DVN model, this creates a single verification trust dependency for each route: an outage can halt delivery, and a compromised sole verifier would undermine message integrity.

Recommended Mitigation: Add a second independent DVN to each route once provider coverage exists on all required chains.

Client: Acknowledged, a second DVN is not available on all chains.

BurraSec: Acknowledged

7.3.3 HyperEVM deploy scripts break if MNEMONIC is used instead of a raw PRIVATE_KEY

Target: [MyOFT.ts](#), [MyHyperLiquidComposer_Recoverable.ts](#)

Severity:

- Impact: Informational
- Likelihood: Low

Description: The HyperEVM deploy scripts call `.toString()` on `hre.network.config.accounts` before passing the result into `new EthersSigner(...)`. This is not type-safe because the repo's Hardhat config explicitly supports two different accounts shapes: a mnemonic object when `MNEMONIC` is set, and a single-element private-key array when `PRIVATE_KEY` is set in [hardhat.config.ts:24](#) and [hardhat.config.ts:31](#). The current implementation works for the private-key flow, but if a user enables mnemonic-based config, `.toString()` will not yield a valid private key and the Hyperliquid helper signer may fail unexpectedly.

Recommended Mitigation: Validate the runtime type of `hre.network.config.accounts` before constructing `EthersSigner`. If these deploy scripts require a raw private key, assert that `accounts` is an array and use the first element explicitly. If mnemonic support is intended, derive the signer through the appropriate Hardhat/ethers signer path instead of coercing the config object to string.

Client: Acknowledged, fails explicitly at deployment time.

BurraSec: Acknowledged

7.3.4 Environment-Specific Secret Files Are Not Git Ignored

Target: [.gitignore](#)

Severity:

- Impact: Informational
- Likelihood: Medium

Description: The repository ignores only the literal `.env` file, but the repo structure and deployment documentation encourage environment-specific files such as `.env.mainnet` and `.env.testnet`, which may contain production private keys. Because these variants are not ignored, an operator could accidentally commit a secrets-bearing env file to version control.

Recommended Mitigation: Broaden the ignore pattern to cover environment-file variants while preserving example files. For example:

```
.env*
!.env*.example
```

Client: Fixed in commit [0850a53](#)

BurraSec: Fix looks good

7.3.5 Missing explicit post-unstake liquidity check in `_lzReceive()`

Target: [MyNativeOFTAdapter.sol](#)

Severity:

- Impact: Informational
- Likelihood: Low

Description: In the auto-unstake branch of `_lzReceive()`, the contract does not explicitly verify `address(this).balance >= amountLD` after calling `STAKING.removeStake(...)` before decrementing `totalDeposited` and attempting the final payout. This is not a security bug, because any later failure in the value-bearing transfer reverts the entire transaction and rolls back the accounting change. However, an explicit post-unstake balance check would make the intended liquidity assumption clearer, improve revert diagnostics, and simplify maintenance.

Recommended Mitigation: Add an explicit post-unstake liquidity check such as `require(address(this).balance >= amountLD, "Adapter: insufficient balance after unstake");` before decrementing `totalDeposited`. This would improve readability and monitoring without changing the security properties of the function.

Client: Acknowledged

BurraSec: Acknowledged

7.3.6 LayerZero shared-decimal quantization introduces a minimum effective bridge amount

Target: [MyNativeOFTAdapter.sol](#)

Severity:

- Impact: Informational
- Likelihood: Medium

Description: LayerZero shared-decimal normalization quantizes bridge amounts to multiples of 1e12 wei (0.000001 token) before transmission. In this system, any non-zero inbound amount is therefore at least 1000 rao, which is relied on by [MyNativeOFTAdapter.sol:258](#). This is expected OFT behavior, but it creates an integration edge case: sub-minimum send amounts are dust-stripped in the OFT base and may revert via slippage protection, or become zero-value no-op messages if callers set `minAmountLD = 0`.

The effect is primarily UX/integration confusion rather than loss of funds or accounting failure.

Recommended Mitigation: Explicitly document the minimum effective bridge amount in user-facing docs and frontend validation, and reject sub-minimum transfers before submitting transactions. If direct task or script usage is expected, enforce `amount >= 1e12 wei` and avoid allowing `minAmountLD = 0` for normal user flows so dust-sized requests fail locally with a clear error instead of producing a no-op cross-chain message.

Client: Acknowledged, Known OFT shared-decimal behavior.

BurraSec: Acknowledged

7.3.7 `registerClaimProxy` missing `msg.value` validation

Target: [MyNativeOFTAdapter.sol](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: The `registerClaimProxy` function allows the owner to register an external account as a RootClaim proxy for the contract's derived coldkey. Registering a proxy on Bittensor requires a deposit of approximately 0.093 TAO, which is refunded when the proxy is removed. The function is marked `payable` and its NatSpec instructs the caller to send `msg.value` to cover this deposit.

However, the function does not validate that `msg.value` matches the required proxy deposit. This creates two problematic scenarios:

- If `msg.value` is insufficient (or zero), the shortfall is silently covered by the contract's existing balance, which belongs to bridge users. When the proxy is later removed via `removeClaimProxy`, the full deposit refund is forwarded to the owner. This means the owner effectively extracts user funds through the deposit/refund cycle.
- If `msg.value` exceeds the required deposit, the surplus remains in the contract's balance. Since it is not tracked by `totalDeposited`, it appears as yield and can be harvested by the owner via `harvestYield`.

Since the owner is a trusted role, the practical risk is minimal. Nevertheless, without proper validation, the function allows the owner to inadvertently (or intentionally) extract value from user deposits.

Recommended Mitigation: Add a `require` check to ensure `msg.value` covers the proxy deposit cost, which can be a state variable in the contract:

```
function registerClaimProxy(bytes32 proxyAccountId32) external payable onlyOwner {
+   require(msg.value == proxyDeposit, "Insufficient proxy deposit");
    PROXY.addProxy(proxyAccountId32, 17, 0);
    emit ClaimProxyRegistered(proxyAccountId32);
}
```

Client: Fixed in commit [4efb0af](#)

BurraSec: Fix looks good